Gatekeeper

System Design Description

Abstract
This document describes the design and implementation of the Gatekeeper support system of Arrowhead (documentation edition 5.3.0) (`ah5-Gatekeeper-java-spring`).

Contents

1 Documentation edition and implementation scope **3**

2 Overview **3**

3 Implementation **4**
3.1 Implementation language and tools 4
3.2 Functional properties implementation 4
3.3 Non functional properties implementation 5
3.4 Deployment 6
3.5 Local modifications 6
3.6 Verification 6

4 Services **7**

5 Go and Rust implementation variants **7**
5.1 Go implementation 7
5.2 Rust implementation 7

6 References **8**

7 Revision History **10**
7.1 Amendments 10
7.2 Quality Assurance 10

1 Documentation edition and implementation scope

This is the 5.3.0 documentation edition, dated 2026-11-09. It carries forward the system and service contracts from the retained 5.2.2 documentation and identifies the current primary implementation sources. The former `documentation/v5.2.2` folder is retained unchanged.

Document version 5.3.0 is distinct from executable, dependency, registered system/service and wire-contract versions. Values still defined by the source (including 5.2.2 artefact versions, 1.0.0 service versions and `SysMonitor-AMQP-IDD-5.2.2` payload identifiers) are preserved. This documentation update does not change runtime behaviour or establish a new implementation release. Historical verification statements describe the tests recorded in the inherited baseline; this revision validates document compilation and references.

2 Overview

This document describes the Gatekeeper system implementation, which realizes the Gatekeeper SysD [1]. The Gatekeeper connects the local cloud to its neighbour clouds through the inter-cloud MQTT relay: global service discovery of authorized foreign providers and inter-cloud negotiation of GateTunnel sessions. In Section 3, we describe implementation details of the system.

3 Implementation

This implementation realizes the Gatekeeper SysD in `ah5-Gatekeeper-java-spring`. It follows the Arrowhead Java-Spring pattern of the other support systems. The Java sources, the private copy of the Arrowhead common library (`vendor/`), the tests, the images and the deployment definition are owned by this folder. The former native implementation (JDK `HttpsServer`, SQLite, package `eu.arrowhead.port`) has been removed. The Python predecessor is preserved in `legacy/` and is excluded from the build.

3.1 Implementation language and tools

- Java 21, Spring Boot 3.4 (web, security, data JPA), the vendored Arrowhead common-utils and data-transfer-objects, Bouncy Castle (PEM relay identity), and the Eclipse Paho MQTT v3 relay client. Maven produces only `arrowhead-gatekeeper-5.2.2.jar`, a Spring Boot JAR. Its entry point `GatekeeperMain` starts Gatekeeper, or with `migrate-sqlite` imports an old store.
- Storage: a private MySQL 8.4 schema `ah_gatekeeper`, through JPA (`ddl-auto=validate`) and the MariaDB driver with `sslMode=verify-full`. The application account has `SELECT`, `INSERT`, `UPDATE` and `DELETE` only.

3.2 Functional properties implementation

The following components implement the system (package `eu.arrowhead.gatekeeper`):

Component	Responsibility
`GatekeeperSystemInfo`	The registered system and services: `globalServiceDiscovery`, `interCloudNegotiation` and `gatekeeperManagement` 5.2.2, and `generalManagement` 1.0.0. The native `sysMonitor` offer is added by the common library.
`api.http.GatekeeperAPI`, `ManagementAPI`, `GeneralManagementAPI`	The HTTP operations, with the native paths, JSON shapes and status codes.
`service.FederationService`	Global service discovery and inter-cloud negotiation, on both the consumer and provider side. Provider-side `authorizationManagement.check-policies` for foreign consumers; GateTunnel sessions; the AH-GT-1 downgrade refusal.
`service.PeerNames`, `FederationSettings`	Peer cloud identities, explicit cross-organisation bindings and the federation profile (checked at start-up).
`relay.RelayClient`, `RelayTls`	The MQTT relay link with its own organisation-signed identity and CRL check. Topics `arrowhead/relay/<cloud>/gatekeeper/in/<sender CN>/<id>`, correlation and duplicate handling are byte-for-byte those of the Go and Rust implementations.
`service.ArrowheadConsumer`	ServiceRegistry lookup and orchestrated consumption through the common `ArrowheadHttpService`.
`security.*`	Token-free liveness, the caller certificate profile (`sy`, `op`), the Gatekeeper caller rules, and the native response headers and `error` field.
`jpa.*`, `migrate.*`	Entities `Cloud`, `Relay` and `Logs`, and the one-off SQLite import.

Every request passes the filter chain in this order:

1. native response headers;
2. liveness (`/health`, token-free);
3. caller certificate profile;
4. the common `OutsourcedFilter`, which verifies the identity token with Authentication and binds it to the client certificate;
5. the common management filter (whitelist policy: LocalCloudManager and the system operator);
6. the Gatekeeper caller rules (inter-cloud and echo operations: the orchestrators, LocalCloudManager, Gatekeeper and the system operator).

3.2.1 Data handled and stored

Storage: MySQL schema `ah_gatekeeper`.

Tables	Content
`cloud`	Neighbour clouds; (name, operator) is unique.
`relay`	Relays; the name is unique.
`logs`	Log entries (Log4j2 JDBC appender), served by `generalManagement.get-log`.

The relay and inter-cloud messages are JSON objects built at run time. Their structure is specified in the IDD documents.

3.3 Non functional properties implementation

3.3.1 Security

Spring Boot TLS uses a PKCS#12 system identity and truststore, with mandatory client certificates (TLS 1.3/1.2) and the cloud CRL (`arrowhead.ssl.crl-file`). Authentication is outsourced: the identity token is bound to the certificate, and only `sy` and `op` certificates are accepted. The `SecurityProfileGuard` refuses to start with TLS off, declared identities or service authorization off. `arrowhead.security.profile=development` is an explicit exception for isolated tests. The relay link uses a separate organisation-signed PEM identity and organisation trust, with CRL checking.

3.3.2 Power management

None; the system is a server process without power states. The `sysMonitor` `power` observation always reports line supply.

3.3.3 Internal monitoring

Logs go to stdout, a rolling file and the `logs` table. The common `SysMonitorPublisher` publishes the `sysMonitor` observations (see the sysMonitor IDD). The echo health is degraded while the relay link is down.

3.3.4 Configuration

Configuration is `config/application.properties`. It reads the environment names of the SysD as `${ENV:default}` placeholders and the secrets through `configtree:/run/secrets/`.

3.4 Deployment

The local `Dockerfile` builds only this folder and runs `arrowhead-gatekeeper-standalone:5.2.2` as UID 10001 with a read-only root filesystem. The local `compose.yaml` starts two services:

- Gatekeeper, on an operator-selected existing cloud network;
- its private database `gatekeeper-db` (MySQL 8.4.11, secure transport required, internal network only). The image creates the schema and restricts the application account to DML.

The HTTPS API uses port 8449. The container health check is a TCP check of that port; the native plain-HTTP health port is gone. Certificates and credentials are mounted individually. This packaging is supplied for operator deployment; existing test-cloud deployments are not changed by it.

ServiceRegistry is the bootstrap endpoint. Authentication and the orchestrator are found there, and ConsumerAuthorization and GateTunnel are consumed through orchestration on behalf of Gatekeeper. Federation additionally requires the organisation MQTT relay, configured neighbour clouds, the relay identity and organisation trust. The Compose default is AH-GT-1, with explicit legacy-mTLS compatibility; peers must agree and never silently downgrade. `migrate-sqlite <file>` imports the clouds and relays of a native SQLite store once, without starting the HTTP listener or registering.

3.4.1 Ownership and build boundary

The Arrowhead common library is a private source copy (`vendor/`, with `PROVENANCE.json`) compiled into this project. The POM inherits only the Spring Boot parent. There is no shared runtime process, data directory or Arrowhead Maven parent. Security fixes to the common library must be propagated to every vendored copy. Build with Java 21 and `mvn -B clean verify`, or use the local Docker build.

3.5 Local modifications

Notes on this implementation in the workspace:

- Inter-cloud orchestration with DynamicServiceOrchestration (local extension); provider-side `check-policies` for foreign consumers.
- Services registered with version `5.2.2`; `generalManagement` 1.0.0 with `status`, `get-log` and `get-config`.
- Deviations from the native implementation: a wrong method answers 405; trailing slashes are not matched; the plain-HTTP health port is removed. Error bodies keep the native `error` field.

3.6 Verification

The project owns these JUnit tests:

- a route contract test: every native path and method through the deployed filter chain, with the authorised, unauthenticated, wrong-certificate, refused-profile, missing-certificate and unauthorised statuses;
- the provider-side `check-policies` tests and the federation protocol steps;
- management validation and views, and peer names;
- the refusal of insecure settings.

Two integration tests run against containers: the whole application against a fresh database image (schema validation under DML-only grants, plus the SQLite import), and a relay round trip against a TLS Mosquitto broker. A live federation run against the Go and Rust Gatekeepers has not been repeated for this packaging.

4 Services

The implementation of the services is based on the SD and IDD documents referenced below.

Services produced	**SysD ref**	**SD ref**	**IDD ref**
`globalServiceDiscovery`	[1]	[2]	[3]
`interCloudNegotiation`	[1]	[4]	[5]
`gatekeeperManagement`	[1]	[6]	[7]
`sysMonitor`	[1]	[8]	[9]
Services consumed	**SysD ref**	**SD ref**	**IDD ref**
serviceDiscovery	[10]	[11]	–
identity	[12]	[13]	–
authorizationManagement	[14]	[15]	–
gateTunnelConnection	[16]	[17]	–
inter-cloud messages	Relay (MQTT)	–	–
sysMonitor (publisher)	AMQP broker (SysMonitor)	–	–

5 Go and Rust implementation variants

5.1 Go implementation

Primary system folder: `Eclipse-Arrowhead-go/ah5-Gatekeeper-go`.

The system owns its Go module, launcher and build configuration. Use the per-system `Makefile` and `go.mod`; the containing Go tree is the build source, not a historical Arrowhead copy. Ordinary native roles use independent identities and JSON snapshot state rather than the Java database. Do not assume Java SQL migration, clustering or identical transaction/retry semantics. The registry has its own configuration and optional native AMQP monitor; other roles must be checked against their own registrations.

Listening port. Built-in default 127.0.0.1:8449 (Arrowhead default). The shipped config.json, compose.yaml and Dockerfile use the same port on all interfaces ("listen": "0.0.0.0:8449"; 8540–8555 layout before 2026-11-09). Override with "listen" in the configuration file, SERVER_PORT=N or -server.port N; the port overrides keep the configured host.

Native behavior and outstanding work. Native Go legacy-mTLS cloud discovery/negotiation engine adapted to this runtime. AH-GT-1 encryption, complete current Java edge-case parity, real peer/broker/ACL qualification. These are the boundaries recorded by the native compatibility document; later implementation changes still require direct qualification. Native design/route evidence: `Eclipse-Arrowhead-go/ah5-Gatekeeper-go/documentation/Gatekeeper`.

For this Java interface, select only a native transport and operation explicitly implemented and advertised by that port. A matching method/path does not establish DTO, error, authorization, retry or persistence equivalence. Java MQTT/AMQP bindings are not automatically provided by a Go or Rust role. Differential contract and security checks remain necessary before substitution.

5.2 Rust implementation

Primary system folder: `Eclipse-Arrowhead-rust/ah5-Gatekeeper-rust`.

The per-system entry point is declared as a binary of `arrowhead-support-rust` in `ah5-support-runtime-rust/Cargo.toml`. Build from the primary Rust workspace with the applicable package/binary selection. Documentation links can resolve into the shared core/support catalogue. Independent process identity and state remain required.

The ordinary native profile uses Tokio/Axum and rustls with peer-identity checks and independent locked JSON state. Network effects are not distributed database transactions. The precise trust, CRL reload and persistence guarantees are those of the native profile, not the Java implementation.

Listening port. Default 127.0.0.1:8449 (Arrowhead default, role gatekeeper). Override with "listen": "host:port" in the configuration, SERVER_PORT=N or –server.port N; the port overrides keep the configured host.

Native behavior and outstanding work. Neighbour/relay administration, actual MQTT global discovery and negotiation with explicit cloud/organisation bindings. Exhaustive DTO/filter/failure parity and independent review. These are the boundaries recorded by the native compatibility document; later implementation changes still require direct qualification. Native design/route evidence: `Eclipse-Arrowhead-rust/ah5-support-runtime-rust/documentation/Gatekeeper`.

For this Java interface, select only a native transport and operation explicitly implemented and advertised by that port. A matching method/path does not establish DTO, error, authorization, retry or persistence equivalence. Java MQTT/AMQP bindings are not automatically provided by a Go or Rust role. Differential contract and security checks remain necessary before substitution.

6 References

[1] J. Delsing, “Gatekeeper – System Description,” Eclipse Arrowhead, SysD 5.3.0, 2026, ah5-Gatekeeper-java-spring/documentation/v5.3.0/Gatekeeper-v5.3.0-SysD.pdf.

[2] ——, “globalServiceDiscovery – Service Description (Gatekeeper),” Eclipse Arrowhead, SD 5.3.0, 2026, gatekeeper-v5.3.0-globalServiceDiscovery-SD.pdf.

[3] ——, “globalServiceDiscovery HTTP/TLS/JSON – Interface Design Description (Gatekeeper),” Eclipse Arrowhead, IDD 5.3.0, 2026, gatekeeper-v5.3.0-globalServiceDiscovery-HTTP-IDD.pdf.

[4] ——, “interCloudNegotiation – Service Description (Gatekeeper),” Eclipse Arrowhead, SD 5.3.0, 2026, gatekeeper-v5.3.0-interCloudNegotiation-SD.pdf.

[5] ——, “interCloudNegotiation HTTP/TLS/JSON – Interface Design Description (Gatekeeper),” Eclipse Arrowhead, IDD 5.3.0, 2026, gatekeeper-v5.3.0-interCloudNegotiation-HTTP-IDD.pdf.

[6] ——, “gatekeeperManagement – Service Description (Gatekeeper),” Eclipse Arrowhead, SD 5.3.0, 2026, gatekeeper-v5.3.0-gatekeeperManagement-SD.pdf.

[7] ——, “gatekeeperManagement HTTP/TLS/JSON – Interface Design Description (Gatekeeper),” Eclipse Arrowhead, IDD 5.3.0, 2026, gatekeeper-v5.3.0-gatekeeperManagement-HTTP-IDD.pdf.

[8] ——, “sysMonitor – Service Description (Gatekeeper),” Eclipse Arrowhead, SD 5.3.0, 2026, gatekeeper-v5.3.0-sysMonitor-SD.pdf.

[9] ——, “sysMonitor AMQP/TLS/JSON – Interface Design Description (Gatekeeper),” Eclipse Arrowhead, IDD 5.3.0, 2026, gatekeeper-v5.3.0-sysMonitor-AMQP-IDD.pdf.

[10] ——, “ServiceRegistry – System Description,” Eclipse Arrowhead, SysD 5.3.0, 2026, ah5-core-java-spring/serviceregistry/documentation/v5.3.0/ServiceRegistry-v5.3.0-SysD.pdf.

[11] ——, “serviceDiscovery – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.3.0, 2026, ah5-core-java-spring/serviceregistry/documentation/v5.3.0/ServiceRegistry-v5.3.0-serviceDiscovery-SD.pdf.

[12] ——, “Authentication – System Description,” Eclipse Arrowhead, SysD 5.3.0, 2026, ah5-core-java-spring/authentication/documentation/v5.3.0/Authentication-v5.3.0-SysD.pdf.

[13] ——, “identity – Service Description (Authentication),” Eclipse Arrowhead, SD 5.3.0, 2026, ah5-core-java-spring/authentication/documentation/v5.3.0/Authentication-v5.3.0-identity-SD.pdf.

[14] ——, “ConsumerAuthorization – System Description,” Eclipse Arrowhead, SysD 5.3.0, 2026, ah5-core-java-spring/consumer-authorization/documentation/v5.3.0/ConsumerAuthorization-v5.3.0-SysD.pdf.

[15] ——, “authorizationManagement – Service Description (ConsumerAuthorization),” Eclipse Arrowhead, SD 5.3.0, 2026, ah5-core-java-spring/consumer-authorization/documentation/v5.3.0/ConsumerAuthorization-v5.3.0-authorizationManagement-SD.pdf.

[16] ——, “GateTunnel – System Description,” Eclipse Arrowhead, SysD 5.3.0, 2026, ah5-GateTunnel-java-spring/documentation/v5.3.0/GateTunnel-v5.3.0-SysD.pdf.

[17] ——, “gateTunnelConnection – Service Description (GateTunnel),” Eclipse Arrowhead, SD 5.3.0, 2026, gateTunnel-v5.3.0-gateTunnelConnection-SD.pdf.

7 Revision History

7.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-25	5.2.2	Independent Java build, private helpers and state, per-system deployment and configuration.	Codex (draft)
3	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): standalone deployment, configuration and security statements updated.	Claude Opus 5.5 (draft)
4	2026-11-09	5.2.2	Implementation converted to the Arrowhead Java-Spring pattern: vendored common library, private MySQL schema (JPA), common authentication, registration, orchestration and sysMonitor; relay, federation protocol and service contracts unchanged.	Claude Opus 5.5 (draft)
5	2026-11-09	5.3.0	New documentation edition; primary-source references and Go/Rust implementation scope; 5.2.2 retained.	Documentation maintenance

7.2 Quality Assurance

No.	Date	Version	Approved by
1		5.3.0	